서 울 대 학 교
SEOUL NATIONAL UNIVERSITY

Systems Software & Architecture Laboratory
Dept. of Computer Science and Engineering

4190.307: Operating Systems
Midterm Exam. (Fall 2024)

12:30 – 13:45, October 22, 2024. Instructor: Jin-Soo Kim

Student ID: _____

Name: _____

Codename: _____

Q1 (40)		Q4 (20)	
Q2 (70)		Q5 (60)	
Q3 (30)		Q6 (120)	
		Total (340)	

I hereby certify that I have neither given nor received aid in answering the questions on this exam. _____
(Signature)

1. Fill in the blank with a single English word (including hyphenated ones). (4 points each)

(1) Operating systems often have a problem known as the ( ) of generality, where they are tasked with general support for a broad class of applications and systems. The fundamental result of it is that the OS is not likely to support any one installation very well.

(2) The developers were also concerned about memory ( ), programs that use a lot of memory and make it hard for other programs to run.

(3) More recently, as the need for better TLB behavior is more common among many applications, Linux developers have added ( ) huge page support. When this feature is enabled, the operating system automatically looks for opportunities to allocate huge pages without requiring application modification.

(4) Unfortunately, things get a little more interesting when little memory is free. In such a case, this memory ( ) forces the OS to start paging out pages to make room for actively-used pages.

(5) FIFO and ( ) clearly do not obey the stack property, and thus are susceptible to anomalous behavior.

(6) The OS could guess that a page is about to be used, and thus bring it in ahead of time; this behavior is known as ( ) and should only be done when there is reasonable chance of success.

(7) Sometimes randomly accessing your address space, particularly if the number of pages accessed exceeds the TLB ( ), can lead to severe performance penalties.

(8) TLB access can easily become a bottleneck in the CPU pipeline, in particular with what is called a(an) ( ) cache.

(9) The general technique of ( ) is commonly used in systems when there is a fixed cost to some operation. By incurring that cost less often, the total cost to the system is reduced.

(10) To implement virtualization of the CPU, and to implement it well, the OS will need both some low-level machinery and some high-level intelligence. We call the low-level machinery ( ).

2. The box on the bottom right shows the initial part of the trap handler that is invoked when a trap occurs while the CPU is executing a user process in xv6. Answer the following questions.

(1) What does the `a0` register hold when the first instruction is about to be executed? (5 points)

```
uservec:
        csrw sscratch, a0
        ...
```

(2) What does the `sscratch` register hold when the first instruction is about to be executed? (5 points)

(3) What is the purpose of the first `csrw` instruction, and why is it necessary? (10 points)

(4) Select all the correct statements about the situation when the first `csrw` instruction is executed. Selecting a wrong answer will cancel out one correct answer. (30 points)

A. The CPU's privilege level is the Supervisor mode.
B. Interrupt is enabled.
C. User page table is being used.
D. The `uservec` routine is stored in the trampoline page.
E. The `sp` register points to the top of the user stack.
F. The `stvec` register holds the address of the `uservec` routine.
G. The `sepc` register holds the address of the instruction that was interrupted by the trap + 4.
H. The `uservec` routine is loaded from the executable file's code region when the process is created.

(5) The xv6 kernel has another trap handler called `kernelvec`, which is triggered when a trap occurs while the CPU is executing in the kernel. Why does xv6 handle traps differently depending on whether they occur in user mode or kernel mode? (20 points)

3. The following pseudo-code provides an overview of how the paging hardware performs address translation, assuming a single-level page table. Here, `PTBR` refers to the starting address of the page table. Each PTE (Page Table Entry) is 8 bytes in size. Note that the present bit (`PTE.Present`) indicates whether the page is in physical memory or on disk. Fill in the blanks to complete the code. (30 points)

```
VPN = (VirtualAddress & VPN_MASK) >> [          ]
(Success, TlbEntry) = TLB_Lookup(VPN)
if (Success == True)     // TLB Hit
    if (CanAccess(TlbEntry.ProtectionBits) == True)
        Offset   = VirtualAddress & OFFSET_MASK
        PhysAddr = (TlbEntry.PFN << SHIFT) | [          ]
        Register = AccessMemory(PhysAddr)
    else
        RaiseException(PROTECTION_FAULT)
else                      // TLB Miss
    PTEAddr = PTBR + [                    ]
    PTE = AccessMemory(PTEAddr)
    if [                                  ]
        RaiseException(SEGMENTATION_FAULT)
    else
        if (CanAccess(PTE.ProtectionBits) == False)
            RaiseException(PROTECTION_FAULT)
        else if (PTE.Present == True)
            // assuming hardware-managed TLB
            TLB_Insert(VPN, PTE.PFN, PTE.ProtectionBits)
            RetryInstruction()
        else if (PTE.Present == False)
            RaiseException(PAGE_FAULT)
```

4. What are the advantages of <u>not</u> using KPTI (Kernel Page Table Isolation), i.e., when the kernel virtual address space is part of the user virtual address space? (20 points)

-

5. Assume a modern paging-based virtual memory system similar to Linux. Let VPN_i be the set of valid virtual page numbers (VPNs) used by process P_i and $m_i(v)$ be a mapping function that returns the page frame number f corresponding to the virtual page $v \in VPN_i$. If there is no corresponding page frame for some $v \in VPN_i$, $m_i(v)$ returns NULL. Now, we can define PFN_i, the set of valid page frame numbers (PFNs) used by Process P_i as follows: $PFN_i = \{ f \mid \forall v \in VPN_i,\ m_i(v) = f\ and\ f \neq NULL \}$. Answer the following questions.

(1) Can we create a process P_i such that $VPN_i = \emptyset$? Explain why. (10 points)

(2) Show whether the following is possible or not for two processes P_i and P_j. If your answer is yes, describe all the possible scenarios. If not, explain why. (40 points)

A. $|VPN_i| > |PFN_i|$

B. $|VPN_i| < |PFN_i|$

C. For some $u, v \in VPN_i\ such\ that\ u \neq v\ and\ m_i(u) = m_i(v) = f\ and\ f \neq NULL$

D. For some $u \in VPN_i,\ v \in VPN_j\ such\ that\ u = v\ and\ m_i(u) = m_j(v) = f\ and\ f \neq NULL$

E. For some $u \in VPN_i,\ v \in VPN_j\ such\ that\ u \neq v\ and\ m_i(u) = m_j(v) = f\ and\ f \neq NULL$

(3) For a given valid page frame $f \neq NULL$, how can we find P_i and v such that $v \in VPN_i\ and\ m_i(v) = f$. List all the possible ways. (10 points)

6. Consider the following `schedtest` program. Assume the followings.

- The `schedtest` program is executed on the xv6 operating system, which runs on a single RISC-V CPU.
- The `nice(int inc)` system call works as described in PA3; it adds `inc` to the nice value of the calling process. A lower nice value indicates a higher priority.
- The `init` and shell (`sh`) processes have a default nice value of zero. When a process is forked, it inherits the nice value of its parent process.
- Each time the `fork()` system call is invoked, the process ID (PID) increases by one. The `init` process and the `sh` process have PIDs of 1 and 2, respectively.
- The `schedtest` process begins its execution at the start of tick 3.
- The `do_nerdy1()` and `do_nerdy2()` functions just perform computations for 1.2 ticks and 2.3 ticks, respectively.

Answer the following questions.

```
// schedtest.c
int main(int argc, char *argv[])
{
  int pid1, pid2, ret;

  pid1 = fork();
  nice(pid1? 10 : 5);
  pid2 = fork();
  if (pid1) {
    if (pid2) {
      nice(-3);
      wait(&ret);
      printf("first ret=%d\n", ret);
      wait(&ret);
      printf("second ret=%d\n", ret);
      exit(1);
    }
    else {
      do_nerdy1(); // takes 1.2 ticks
      exit(2);
    }
  }
  else if (pid2)
    exit(3);
  else {
    do_nerdy2();   // takes 2.3 ticks
    exit(4);
  }
}
```

(1) How many processes are created when the `schedtest` program is executed, including the `schedtest` process itself? (5 points)

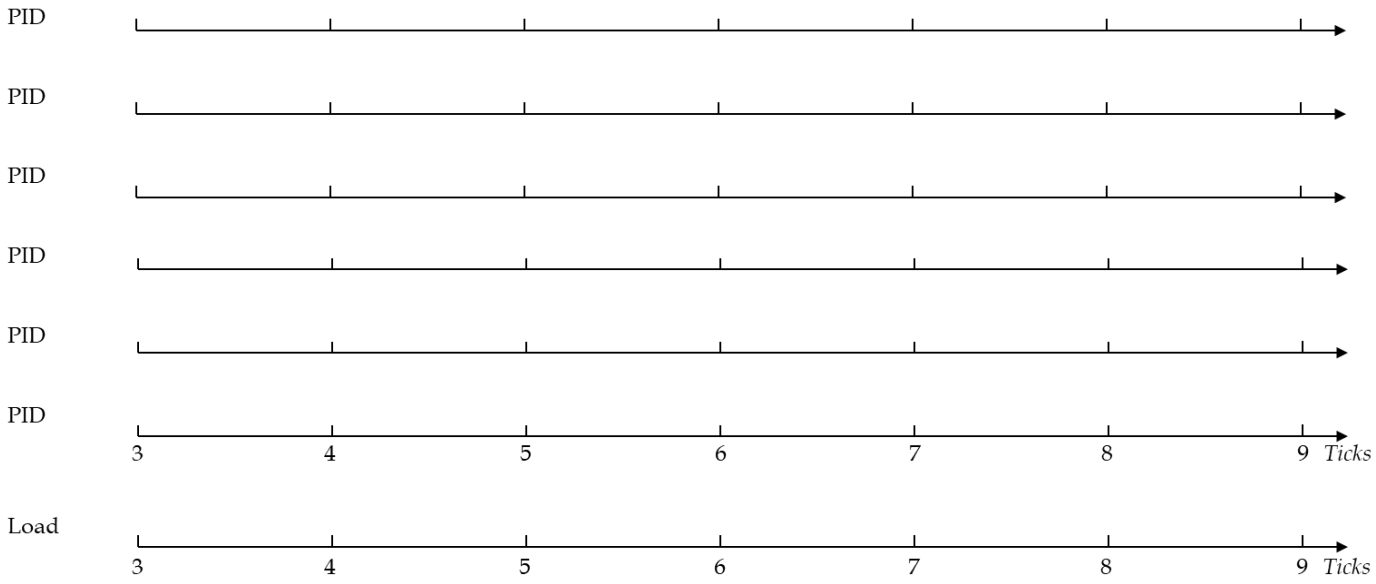
(2) Draw a diagram that illustrates the hierarchy of processes (except for `init` and `sh`). Use circles to represent each process and show the parent-child relationships with connecting lines. Label each process with its process ID (PID) and its final nice value. (25 points)

(3) The following is the output from the `schedtest` process. Fill in the blanks with the appropriate values. (10 points)

```
first ret=_____________

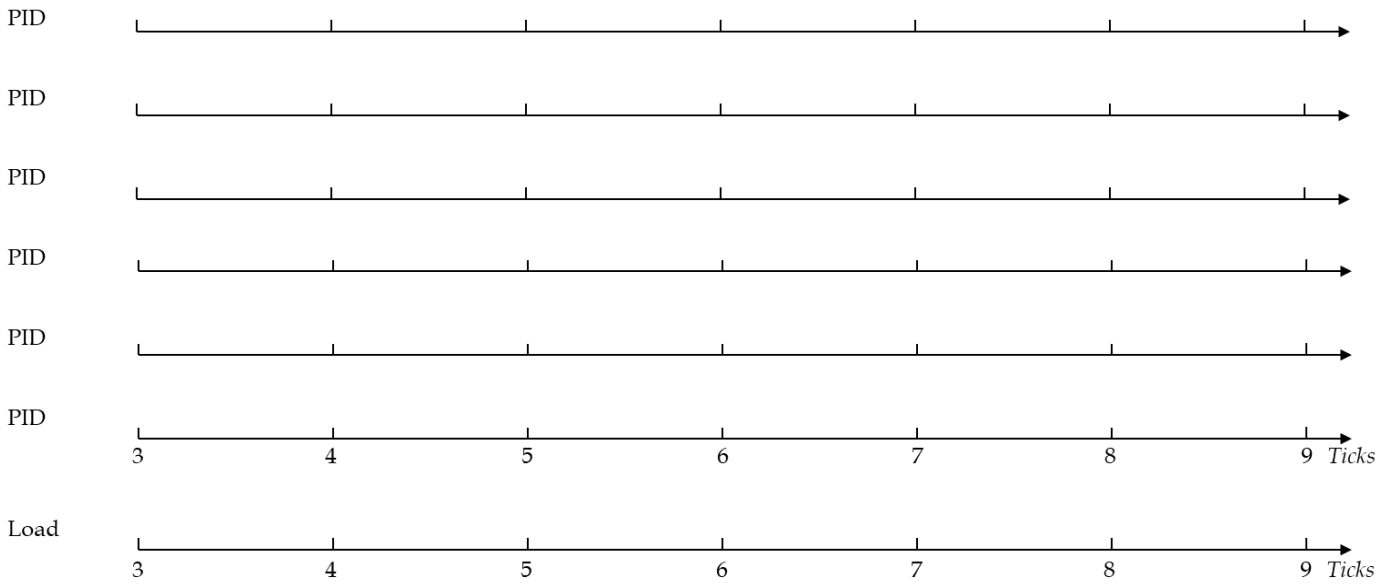
second ret=_____________
```

(4) Complete the following diagram that shows the timing and order in which processes are executed over time under the default xv6 scheduler. Also, show the variation in the system load, defined as the number of runnable processes excluding the currently running process. You don't need to illustrate the shell process (PID 2). (40 points)



(5) Complete the following diagram that shows the timing and order in which processes are executed over time under the SNULE scheduler that implements Part 2 of PA3. Show the variation in the system load as well. You don't need to illustrate the shell process (PID 2). (40 points)

Note: *If multiple processes have the highest priority, the process with the smallest PID is scheduled first.*



cf.) Main characteristics of the SNULE scheduler (Part 2):

There are two run queues: the current RQ and the next RQ. New or woken-up processes are placed in the current RQ.

A process with the highest priority in the current RQ is chosen for execution. If it uses up its time slice, it is moved to the next RQ.

When no runnable processes are left in the current RQ, the current and next RQs are swapped.

A process with a lower nice value has a higher priority.

The time slice depends on the system load: 1 tick if load is >= 6, 10 ticks if load is <= 1, and 10/load ticks otherwise.